

Assignment 1: Addition of two integers

Instructor: Subhamoy Moitra

Written by: Sreejit Maity

1 Introduction

As soon as we entered the field of counting favorite things in our early childhood, we came across the idea of Addition. For us, in the basic hours, addition was nothing more than an extended counting with our fingers.

It was an amazing thing to be familiar with. As we grew, we learned to methodically add two integers. But why are we so obsessed with addition and again why does the added sum match exactly with the sum we counted with our fingers?

The reason that we are emphasizing on addition so much, is that is the fundamental binary operation that is used to derive other operations like Multiplication, exponential operations and many more algorithms. The root of the Mathematics part we see happen, lies deep into the basic binary system and logic gates.

2 Addition recap

Let us recapitulate how addition of two integers is generally done. We pick two integers and write them one below another aligning them to their respective place values. Then, we start to add the digits from rightmost end: whenever the sum is less than 9, we write that digit directly; if it exceeds 9, we write the unit place digit and take carry of 1 in the next two digits' addition. In this way, the addition of two integers is done.

Now, we will visualize, how addition is done using binary numbers and logic gates.

3 Full Adder

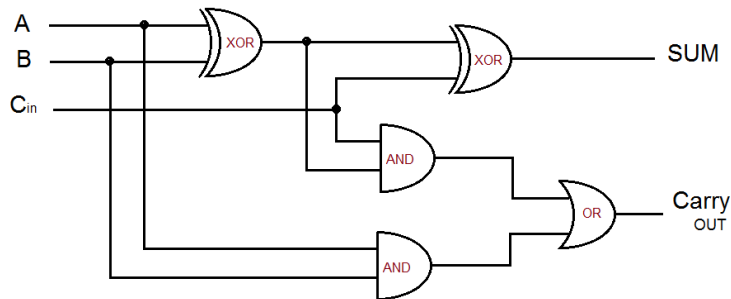
Full Adder is basically a logic circuit that enables the addition of three binary inputs, say A , B and C_{in} . Full adder provides two outputs: SUM (S) and CARRY (C_{out}). Using these, we can find the addition of the integers, given in bits format.

The Truth Table for Full Adder is:

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Truth table for a full adder

The circuit diagram is given by:



4 Addition of two n-bit integers using Full adders

A single full adder can only add one bit position at a time. To add two n -bit numbers

$$A = A_{n-1}A_{n-2} \cdots A_1A_0, \quad B = B_{n-1}B_{n-2} \cdots B_1B_0,$$

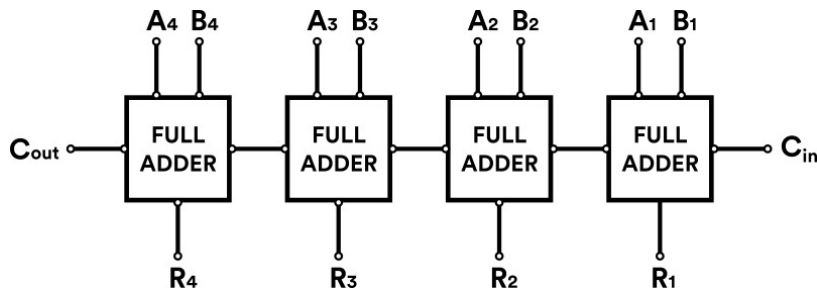
we use n full adders, one per bit position, and connect them so that the carry-out of each stage becomes the carry-in of the next. Concretely, for $i = 0, 1, \dots, n-1$,

$$(S_i, C_{i+1}) = \text{FA}(A_i, B_i, C_i),$$

with the boundary condition $C_0 = 0$, since there is no bit before position 0 to generate an incoming carry. The final carry C_n , produced by the last full adder, is the overall carry-out of the addition (used to detect overflow or as an extra output bit).

This wiring pattern, where the carry ripples from the least significant bit to the most significant bit, is called a ripple carry adder. The key point is that each full adder cannot compute its outputs until it knows the carry C_i coming from the previous stage, so the stages must be evaluated, at least logically, in order from $i = 0$ up to $i = n-1$.

The diagram for addition of 4-bit integers is given below:



5 Reference

I have taken the Truth table from Claude and the two diagrams from Google images. Other details relate from class notes and ideas.